

Guía Paso a Paso – Evaluación de API Insegura

1. Preparar el entorno

Asegúrate de tener **Python 3.9+** instalado y FastAPI disponible.

🌱 **Instala las dependencias:**

```
pip install fastapi uvicorn
```

2. Crea los archivos del proyecto

Estructura recomendada:

api_vulnerable/

├── main.py

├── models.py

└── schemas.py

3. Ejecutar la API

En consola:

```
uvicorn main:app --reload --port 3000
```

Accede a la documentación Swagger:

📌 <http://localhost:3000/docs>

4. Realizar las pruebas desde Swagger

✓ Prueba 1: GET `/api/users`

- Haz clic en `GET /api/users`
- Pulsa "Execute"
- 📌 Verás los usuarios **con emails y contraseñas visibles**
- Observa que no pide autenticación

✓ Prueba 2: DELETE `/api/users/1`

- Ejecuta `DELETE` sobre el usuario 1
- 📌 No hay token ni rol requerido, y el usuario se elimina
- Observa el fallo de autorización crítica

✓ Prueba 3: POST `/api/users`

En el body escribe:

```
{  
  "username": "<script>alert('XSS')</script>",  
  "email": "notanemail"  
}
```

- 📌 Se acepta el payload sin validación ni sanitización

5. Exportar y probar desde Postman

Descarga la especificación OpenAPI

Visita:

<http://localhost:3000/openapi.json>

Guarda como `openapi.json`.

Importar en Postman

1. Abre Postman
2. Haz clic en "Import"
3. Selecciona el archivo `openapi.json`
4. Se generarán los endpoints automáticamente

6. Crear manualmente cada endpoint en Postman

Método	URL	Body	Autenticación
GET	http://localhost:3000/api/users	—	Ninguna
POST	http://localhost:3000/api/users	JSON	Ninguna
DELETE	http://localhost:3000/api/users/1	—	Ninguna

Ejemplo para **POST**:

```
{  
  "username": "<img src='x' onerror='alert(1)'>",  
  "email": "no-email"  
}
```

7. Identificar vulnerabilidades (para tu informe)

- ❌ No hay autenticación ni JWT
- ❌ Cualquier usuario puede eliminar sin privilegios
- ❌ Se aceptan scripts XSS y correos inválidos
- ❌ Contraseñas expuestas en texto plano
- ❌ No hay validación ni tipo de datos

8. Recomendaciones a proponer

- ✅ Implementar autenticación JWT
- ✅ Control de roles (RBAC)
- ✅ Validación estricta (pydantic o class-validator)
- ✅ Ocultar contraseñas (no devolverlas)
- ✅ Agregar cabeceras seguras